Diseño de Compiladores I – Compiladores e Intérpretes
Trabajo Práctico Nº 4
Fecha de Entrega: 13/11/2026

Objetivo

Generar código Assembler, a partir del código intermedio generado en el Trabajo Práctico Nº 3.
El mecanismo a utilizar para la generación de la salida será el de **variables auxiliares**.

El código Assembler generado debe poder ser ensamblado y ejecutado sin errores.

Los grupos podrán optar entre la generación de:

- Assembler para Pentium de 32 bits. El ensamblador a utilizar se puede descargar 4del Aula Virtual ,desde la sección Herramientas para el Trabajo Práctico 4.
    - Para las operaciones entre datos de tipo entero se deberá generar código que utilice los registros del procesador (EAX, EBX, ECX Y EDX o AX, BX, CX y DX.
    - Para las operaciones entre datos de punto flotante se deberá utilizar el co-procesador 80X87.
- WebAssembly. https://webassembly.org/

Controles en Tiempo de Ejecución

Incorporar los chequeos en tiempo de ejecución asignados al grupo**.** Estos chequeos controlarán situaciones de error que sólo pueden detectarse en tiempo de ejecución. El código generado por el compilador deberá, cada vez que se produzca la situación de error correspondiente, emitir un mensaje de error, y finalizar la ejecución.

a) División por cero para datos enteros y de punto flotante:
El código Assembler deberá chequear que el divisor sea diferente de cero antes de efectuar una división. Si esto sucede, se deberá emitir un mensaje de error y terminar la ejecución.
Este chequeo deberá efectuarse para el o los tipos de datos asignados al grupo.

b) Overflow en sumas de enteros:
El código Assembler deberá controlar el resultado de la operación indicada, para los tipos de datos enteros asignados al grupo. Si el mismo excede el rango del tipo del resultado, se deberá emitir un mensaje de error y terminar la ejecución.

c) Overflow en sumas de datos de punto flotante:
El código Assembler deberá controlar el resultado de la operación indicada, para el tipo de datos de punto flotante asignado al grupo. Si el mismo excede el rango del tipo del resultado, se deberá emitir un mensaje de error y terminar la ejecución.

d) Overflow en productos de enteros:
El código Assembler deberá controlar el resultado de la operación indicada, para los tipos de datos enteros asignados al grupo. Si el mismo excede el rango del tipo del resultado, se deberá emitir un mensaje de error y terminar la ejecución.

e) Overflow en productos de datos de punto flotante:
El código Assembler deberá controlar el resultado de la operación indicada, para el tipo de datos de punto flotante asignado al grupo. Si el mismo excede el rango del tipo del resultado, se deberá emitir un mensaje de error y terminar la ejecución.

f) Resultados negativos en restas de enteros sin signo:
El código Assembler deberá controlar el resultado de la operación indicada. Este control se aplicará a operaciones entre enteros sin signo. Si una resta entre datos de este tipo arroja un resultado negativo, se deberá emitir un mensaje de error y terminar la ejecución.

g) Pérdida de información en conversión de punto flotante a entero:
El código Assembler deberá controlar el resultado de la operación indicada. Este control se aplicará a conversiones (implícitas o explícitas) de datos de punto flotante a entero. Si la conversión produce pérdida de información, se deberá emitir un mensaje de error y terminar la ejecución.

h) Asignación inválida a variables de tipo enumeración:
El código Assembler deberá controlar que el valor o expresión asignado a una variable de tipo enumeración esté dentro de los valores de la enumeración. Si esto no ocurre, se deberá emitir un mensaje de error y terminar la ejecución.

i) Recursión en invocaciones de funciones

El código Assembler deberá controlar que una función no pueda invocarse a sí misma. Si esto sucede, se deberá emitir un mensaje de error y terminar la ejecución.

GRUPO	CHEQUEOS
1	a b e f
2	b c g i
3	d f g h
4	c d f h
5	a b e g
6	b c g h
7	a d e i
8	a c d i
9	b e h i
10	a b c i
11	d e f g
12	a c d i
13	a b e i
14	a b c i
15	a d e i
16	a c d f
17	b f g h
18	b c g h
19	d e f g
20	c d g i
21	b e g h
22	b f g h
23	d e g h

GRUPO	CHEQUEOS
24	c d g i
25	a b e h
26	a b c h
27	d e h i
28	a c d h
29	b e h i
30	b c g h
31	a d e h
32	c d h i
33	b e f h
34	a b c h
35	d e f g
36	c d f i
37	a b e i
38	b c f h
39	d e f h
40	a c d g
41	a d e g
42	a c d i
43	b e f h
44	b c h i
45	a b e i

Chequeo de Tipos y Conversiones

Para todos los grupos:

Tanto para conversiones explícitas como para conversiones implícitas (agregadas por el compilador), el Assembler debe contener el código correspondiente a dichas conversiones.

Tema 33 y 34: *Conversiones Explícitas*

Los grupos cuyos lenguajes incluyen conversiones explícitas, deberán traducir el código de las conversiones a código Assembler.
El compilador debe prohibir cualquier operación (expresión, asignación, comparación, etc.) entre operandos de tipos diferentes, informando en cada caso, cuál es la combinación de tipos que está provocando la incompatibilidad.

- Los grupos que debieron efectuar el chequeo de tipos durante la etapa 3 (Tercetos y Árbol Sintáctico), ya tienen asegurado que las operaciones a generar se efectuarán con operandos compatibles.
- Los grupos que deben considerar el chequeo de tipos en esta etapa (Traducción de Polaca Inversa a Assembler), deberán chequear que los operandos sean de tipos compatibles durante la traducción de Polaca Inversa a Assembler, informando el error de incompatibilidad de tipos cuando no se dé dicha compatibilidad.

Tema 35: *Conversiones Implícitas*

Para los grupos que deben considerar conversiones implícitas, el Assembler deberá incluir código para efectuar dichas conversiones cuando los tipos de los datos involucrados en una operación lo requieran, y siempre que el lenguaje lo permita.

- Los grupos que debieron generar las conversiones en la representación intermedia (Tercetos y Árbol Sintáctico) deberán traducir, en esta etapa, dichas conversiones a código Assembler.
- Los grupos que deben considerar el chequeo de tipos y/o la incorporación de conversiones en esta etapa (Traducción de Polaca Inversa a Assembler), deberán agregar código para efectuar las conversiones cuando las mismas sean requeridas, siempre que el lenguaje lo permita. Si hubiera incompatibilidad de tipos, esto deberá ser informado por el compilador como un error.

Salidas del compilador

1) Los errores generados en tiempo de compilación (Errores Léxicos, Sintácticos y Semánticos) se deberán mostrar todos juntos, indicando la descripción de cada error y la línea en la que fue detectado.
2) Representación intermedia, según indicaciones del Trabajo Práctico 3.
3) Contenidos de la Tabla de Símbolos.
4) Archivo conteniendo el código Assembler.

Nota: No se deberán mostrar los tokens ni las estructuras sintácticas que se pidieron como salida de los Trabajos Prácticos 1 y 2, a menos que exista una reentrega pendiente que así lo requiera.

Forma de entrega

Se deberá entregar:

- **Código fuente completo y ejecutable, incluyendo librerías del lenguaje y todo otro componente que fuera necesario para la ejecución.**
- **Casos de Prueba:**
  - Casos válidos para cada estructura sintáctica solicitada
  - Casos semánticamente válidos
  - Casos con errores por Tipos incompatibles, y/o que requieran conversiones para los temas 33 al 35.
  - Casos con y sin errores semánticos por redeclaración o no declaración de variables, funciones, clases, atributos, métodos.
  - Casos que violen las restricciones de alcance del lenguaje.
  - Casos de prueba para cada tema particular asignado:
    - Sentencias de control (sentencia IF y temas 11 a 14)
      Incluir en las pruebas, el anidamiento de sentencias de selección e iterativas.
    - Asignaciones en expresiones (temas 17 y 18) con y sin errores semánticos.
    - Funciones con invocaciones y retornos con características especiales (temas 19 a 21) con y sin errores semánticos que puedan detectarse en cada caso.
    - Tema 22: casos válidos e inválidos de uso de variables declaradas como comptime.
    - Tema 23: asignaciones válidas e inválidas a variables declaradas como enumeraciones.
    - Temas 24: Declaraciones de clases con código con la misma estructura y con estructuras diferentes.
    - Temas 24 y 25: Asignaciones válidas e inválidas entre objetos de clases diferentes.
    - Tema 26: Declaraciones de clases con import/export válidas e inválidas.
    - Tema 27: Uso de atributos o métodos desde clases amigas válidos e inválidos.
    - Temas 28 y 29: Acceso correcto e incorrecto a atributos.
    - Temas 30 a 32: Ejemplos donde se deba acceder a atributos y/o métodos de clases heredadas, donde sea necesario desambiguar a qué clase pertenece el método o atributo en cuestión.
  - Casos que generen errores en tiempo de ejecución (para cada control asignado al grupo).
  - Todo otro caso que consideren necesario.
- **Informe conteniendo:**
  - Introducción
  - Descripción de la Generación de Código Intermedio, indicando:
    - estructura utilizada para el almacenamiento del código intermedio,
    - uso de notación posicional de Yacc ($$, $n) o equivalente en SLY, indicando en qué casos se usó, y con qué fin,
    - descripción tipo pseudocódigo del algoritmo usado para la generación de las bifurcaciones en sentencias de control,
    - nuevos errores considerados,
    - y todo otro aspecto que se considere relevante.
  - Descripción del proceso de Generación de Código Assembler, indicando:
    - Mecanismo utilizado para la generación del código Assembler
    - Mecanismo utilizado para efectuar cada una de las operaciones aritméticas
    - Mecanismo utilizado para la generación de las etiquetas destino de las bifurcaciones.
    - Todo otro aspecto que se considere relevante
  - Modificaciones a las etapas anteriores, si hubieran existido.
  - Para los **temas particulares asignados al grupo**, incluir una descripción del modo en que fue resuelto el tema correspondiente en cada etapa de desarrollo del compilador.
  - Si utilizaron IA para generar alguna parte del compilador indicar:
    - Qué herramienta de IA utilizaron
    - Promps utilizados
    - Indicación de la o las partes del compilador que generaron usando IA
    - **Aclaración**: En la defensa, deberán explicar el desarrollo completo del Compilador, justificando por qué consideraron apropiadas la o las partes de la solución del trabajo generadas usando IA
  - Conclusiones